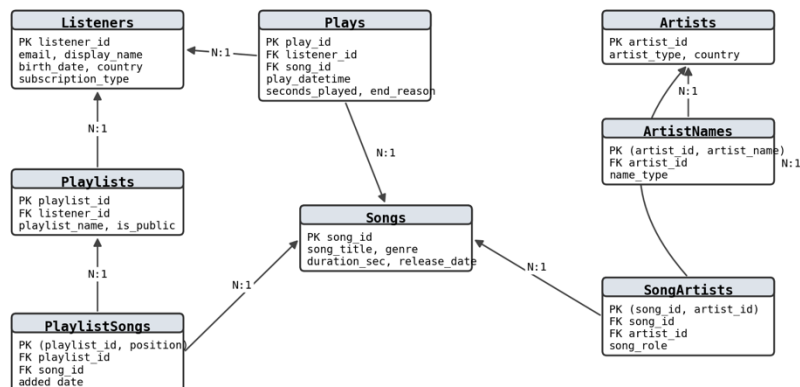


שאלה 3 - עיצוב בסיס נתונים ל-Spotify (צד המאזין)

הנחות שלקחתי

1. מעצב את צד המאזין בלבד. העלאת שירים ותמלוגים אינם מיוצגים.
2. הקטלוג - אמנים, שירים וסוגים - נכתב על ידי מערכת חיצונית מול חברות התקליטים. המאזין קורא ממנו ואינו כותב אליו.
3. סיסמאות ואמצעי תשלום יושבים במערכת נפרדת. כאן נשמרת רק דרגת המנוי, כי היא משפיעה על ההאזנה.
4. אירוע האזנה נרשם פעם אחת בסיום ההשמעה ואינו מתעדכן.
5. לכל שיר סוג ראשי אחד. שיר שמשייך לכמה סוגים - ראו את ההרחבה ב-3.2.

תרשים הסכמה



החץ מצביע מהטבלה שמחזיקה את המפתח הזר אל טבלת האב.

שמונה טבלאות: ארבע ישויות - מאזין, אמן, שיר ורשימה - טבלת שמות אחת, שתי טבלאות קישור וטבלת אירועים. כל אחת נובעת מיחס רבים-לרבים או מדרישה מפורשת בשאלה.

3.1 - קוד יצירת הטבלאות

```
CREATE TABLE Listeners (
  listener_id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE, -- זהות הכניסה
  display_name VARCHAR(100) NOT NULL,
  birth_date DATE NOT NULL, -- ולא גיל; ראו 3.2
  country VARCHAR(50) NOT NULL,
  signup_date DATE NOT NULL,
  subscription_type VARCHAR(10) NOT NULL,
  CONSTRAINT chk_ls_sub CHECK (subscription_type IN ('free','premium')),
  CONSTRAINT chk_ls_birth CHECK (birth_date < signup_date)
);
```

```
-- ArtistNames-אין כאן עמודת שם: כל שמות האמן יושבים ב
CREATE TABLE Artists (
  artist_id INT AUTO_INCREMENT PRIMARY KEY,
  artist_type VARCHAR(10) NOT NULL,
  country VARCHAR(50),
  CONSTRAINT chk_ar_type CHECK (artist_type IN ('solo','band'))
);
```

```
CREATE TABLE ArtistNames (
  artist_id INT NOT NULL,
  artist_name VARCHAR(100) NOT NULL,
  name_type VARCHAR(20) NOT NULL,
  PRIMARY KEY (artist_id, artist_name),
  CONSTRAINT chk_an_type
    CHECK (name_type IN ('main','latin','hebrew',
                        'nickname','surname')),
  FOREIGN KEY (artist_id) REFERENCES Artists(artist_id)
);
```

```
-- החיפוש רץ על העמודה הזו, ולכן היא מאונקסט
CREATE INDEX idx_artist_name ON ArtistNames(artist_name);
```

```
CREATE TABLE Songs (
  song_id INT AUTO_INCREMENT PRIMARY KEY,
  song_title VARCHAR(200) NOT NULL,
  genre VARCHAR(50) NOT NULL, -- הסוג; ראו 3.2
);
```

```

duration_sec INT NOT NULL, -- דרוש לחישוב נטישה
release_date DATE NOT NULL,
CONSTRAINT chk_sg_duration CHECK (duration_sec > 0)
);
CREATE INDEX idx_song_title ON Songs(song_title);
CREATE INDEX idx_song_genre ON Songs(genre);

-- שיר מבוצע בדואט, ולכן היחס בין שיר לאמן הוא רבים לרבים
CREATE TABLE SongArtists (
    song_id INT NOT NULL,
    artist_id INT NOT NULL,
    song_role VARCHAR(10) NOT NULL, -- main / featured
    PRIMARY KEY (song_id, artist_id),
    CONSTRAINT chk_sa_role CHECK (song_role IN ('main','featured')),
    FOREIGN KEY (song_id) REFERENCES Songs(song_id),
    FOREIGN KEY (artist_id) REFERENCES Artists(artist_id)
);

CREATE TABLE Playlists (
    playlist_id INT AUTO_INCREMENT PRIMARY KEY,
    listener_id INT NOT NULL,
    playlist_name VARCHAR(100) NOT NULL,
    is_public CHAR(1) NOT NULL DEFAULT 'N',
    created_date DATE NOT NULL,
    -- ראו 3.5 על UNIQUE בכוונה אין
    CONSTRAINT chk_pl_public CHECK (is_public IN ('Y','N')),
    FOREIGN KEY (listener_id) REFERENCES Listeners(listener_id)
);

CREATE TABLE PlaylistSongs (
    playlist_id INT NOT NULL,
    position INT NOT NULL, -- המקום ברשימה
    song_id INT NOT NULL,
    added_date DATE NOT NULL,
    PRIMARY KEY (playlist_id, position),
    CONSTRAINT chk_ps_position CHECK (position > 0),
    FOREIGN KEY (playlist_id) REFERENCES Playlists(playlist_id),
    FOREIGN KEY (song_id) REFERENCES Songs(song_id)
);

-- פריט המידע שבחרתי: האזנה לשיר ואופן סיומה
CREATE TABLE Plays (
    play_id INT AUTO_INCREMENT PRIMARY KEY,
    listener_id INT NOT NULL,
    song_id INT NOT NULL,
    play_datetime DATETIME NOT NULL,
    seconds_played INT NOT NULL, -- כמה באמת נשמע
    end_reason VARCHAR(10) NOT NULL, -- finished/skipped/stopped
    CONSTRAINT chk_pa_seconds CHECK (seconds_played >= 0),
    CONSTRAINT chk_pa_reason
        CHECK (end_reason IN ('finished','skipped','stopped')),
    FOREIGN KEY (listener_id) REFERENCES Listeners(listener_id),
    FOREIGN KEY (song_id) REFERENCES Songs(song_id)
);
CREATE INDEX idx_plays_song ON Plays(song_id, play_datetime);

```

למה כל CHECK נפרד ובעל שם: פירקתי כל כלל לאילוף עצמאי במקום אילוף אחד גדול עם AND. כשמשוהו נכשל, ההודעה מציינת את שם האילוף ואפשר לראות מיד איזה כלל נשבר.

3.2 - החלטות עיצוב

Listeners - המאזין

מה זה: המשתמש הרשום. שורה אחת לכל מאזין.

שדות: דוא"ל, שם לתצוגה, תאריך לידה, מדינה, תאריך הרשמה וסוג מנוי.

למה תאריך לידה ולא גיל: גיל מתיישן בכל שנה ומחייב עדכון תקופתי של כל השורות. התאריך נכון לתמיד, והגיל נגזר ממנו בשאילתה. הוא נדרש לתוכן מוגבל גיל.

למה מדינה: הקטלוג, המצעדים והתוכן המומלץ שונים בין מדינות, ולכן היא נדרשת להתאמת מה שהמאזין רואה.

מה בכוונה לא נשמר: סיסמה ואמצעי תשלום - מערכת נפרדת. מגדר ומיקום מדויק - אינם משרתים אף אחד מהשימושים שהשאלה מונה. שדה שאינו משרת שימוש הוא חשיפה ולא נכס.

מפתחות: מזהה פנימי. הדוא"ל ייחודי ויכול היה לשמש כמפתח, אבל הוא ניתן לשינוי: אילו Playlists ו-Plays היו מחזיקות אותו כמפתח זר, כל שינוי כתובת היה גורר עדכון בשתייהן. לכן מזהה פנימי, והדוא"ל מוגדר UNIQUE.

יחסים: מאזין אחד - הרבה רשימות והרבה האזנות (1:N מהן אליו).

Artists ו-ArtistNames - דרישה 2: איתור בשמות שונים

הבעיה והפתרון: אמן אחד נושא כמה שמות, ועמודת שם אחת בטבלת האמנים הייתה מאפשרת רק אחד מהם - מי שיחפש "מרקורי" לא היה מוצא את פרדי מרקורי. לכן השם אינו תכונה של האמן אלא ישות: ArtistNames מחזיק שורה לכל שם, והחיפוש רץ על טבלה אחת ומחזיר את אותו artist_id בכל אחד מהם.

```
SELECT DISTINCT artist_id
FROM ArtistNames
WHERE artist_name = 'מרקורי';
```

סוגי השם: name_type מכסה אחד לאחד את המקרים שבשאלה: latin ללועזי, hebrew לתעתיק, surname לשם משפחה ו-nickname לכינוי. main מסמן את שם התצוגה וחופף לרוב ל-latin; הפרדה מלאה הייתה דורשת עמודה נפרדת לשפה ולתפקיד, ולא שווה זאת כאן.

למה אין עמודת שם גם ב-Artists: היא הייתה נוחה לתצוגה, אבל אותו מידע היה יושב בשני מקומות ועלול לסתור את עצמו. מקור אמת אחד. השם הראשי מסומן ב-main = 'name_type'.

המחיר: כל שאילתה שמציגה שם אמן צריכה JOIN. פתרתי בעזרת VIEW אחד שכל השאילתות משתמשות בו:

```
CREATE VIEW artist_display AS
SELECT a.artist_id, a.artist_type, n.artist_name
FROM Artists a
JOIN ArtistNames n ON n.artist_id = a.artist_id
WHERE n.name_type = 'main';
```

מפתחות: לאמן מזהה פנימי - שם אינו מזהה, יש כמה אמנים בשם John Williams. ב-ArtistNames המפתח טבעי: הצירוף של אמן ושם.

אילוצים: המפתח מונע שאותו שם יירשם פעמיים לאותו אמן, ומפתח זר מוודא שהאמן קיים. מה שלא נאסף: שלכל אמן יהיה בדיוק שם ראשי אחד - זה דורש ספירת שורות, וראו הטבלה בהמשך.

Songs ו-SongArtists - דרישה 4: איתור שירים ודואטים

מה זה ומה בו: השירים - שם, אורך בשניות ותאריך יציאה - וטבלת קישור לאמנים שביצעו אותם. האורך אינו קישוט: הוא מה שמאפשר לדעת אם האזנה של ארבעים שניות היא נטישה או שיר קצר שהסתיים.

הערה: לא תמיד תאריך היציאה המדויק ידוע למערכת בעת העלאת השיר.

למה טבלת קישור ולא שתי עמודות אמן: מספר המבצעים אינו קבוע - יש שיר של אמן אחד, דואט, ושיתוף פעולה של חמישה. שתי עמודות היו חוסמות את המקרה השלישי ומשאירות עמודה ריקה בראשון.

למה יש עמודת תפקיד: ההבדל בין אמן ראשי לאורח מופיע בתצוגה ("A feat. B") וגם בשאילתות: "השירים של שאקירה" בדרך כלל מתכוון לשירים שבהם היא ראשית.

הערה: זוהי בחירה עיצובית שהרחיבה את הדרישות, אך במבחן עדיף להיצמד לדרישות ולא להרחיב ללא צורך.

מפתחות: לשיר מזהה פנימי - שני שירים שונים יכולים לשאת אותו שם. בטבלת הקישור המפתח טבעי: הצירוף של שיר ואמן.

יחסים: שיר לאמן - רבים לרבים, ולכן טבלת קישור. שיר לרשימת השמעה - גם רבים לרבים.

הסוג - עמודה ולא טבלה

דילמה עיצובית: האם לייצג סוג כעמודה או כטבלה. האפשרות שבחרתי היא עמודה בטבלת השירים, בהתאם להנחה שלכל שיר סוג ראשי אחד. חיפוש לפי סוג הוא תנאי פשוט על עמודה מאונקסת, בלי JOIN.

החלופה שנדחתה: טבלת Genres בת עמודה אחת עם טבלת קישור SongGenres. היא מרוויחה מפתח זר שמונע "Rock" מול "rock", ומאפשרת כמה סוגים לשיר. ויתרתי עליה כי הקטלוג נכתב על ידי מערכת חיצונית אחת עם רשימת סוגים סגורה. אם יידרשו כמה סוגים לשיר, התוספת היא טבלת הקישור הזו והעמודה genre יורדת - שום דבר אחר בסכמה לא משתנה.

Playlists ו-PlaylistSongs - דרישה 5

ההחלטה המרכזית - המפתח: רשימות ההשמעה של המאזין והשירים שבהן לפי סדרם. המפתח הוא (playlist_id, position) ולא (playlist_id, song_id), כי ברשימה מותר לאותו שיר להופיע פעמיים והסדר משמעותי. מפתח לפי שיר היה אוסר את החזרה; מפתח לפי מיקום אוסף במקומו שבכל מקום ברשימה יושב שיר אחד.

הערה: פתרון זה מוסיף ערך לחוויית המשתמש אך לא נרמז בדרישות השאלה.

אילוצים: מפתחות זרים לרשימה ולשיר, ו-CHECK שהמיקום חיובי. מה שלא נאסף: שהמיקומים יהיו רצף שלם בלי חורים - זה דורש ספירה. חורים אינם מזיקים, כי התצוגה ממיינת לפי position ולא סופרת אותו.

יחסים: מאזין אחד - הרבה רשימות. רשימה אחת - הרבה שירים, ושיר אחד נמצא בהרבה רשימות (רבים לרבים).

Plays - דרישה 6: פריט המידע שבחרתי

מה בחרתי: האזנה לשיר יחד עם אופן סיומה - seconds_played ו-end_reason. כלומר גם ההאזנה וגם הנטישה, שהשאלה מונה כשתי דוגמאות, נשמרות באותה שורה.

למה דווקא הוא: הוא נאסף על כל האזנה בלי שהמאזין עושה דבר, בעוד Likes מסמנים רק חלק קטן מהשירים. והוא היחיד שנושא אות שלילי: היעדר לייק אינו אומר דבר, ונטישה אחרי שש שניות אומרת "לא זה".

שדות ומפתחות: מי, מה, מתי, כמה שניות נשמעו ואיך זה נגמר. המפתח פנימי, כי אין צירוף טבעי "יחודי" - אותו מאזין שומע את אותו שיר שוב ושוב.

אילוצים: CHECK על מספר שניות אי-שלילי ועל ערכי end_reason. מה שלא נאסף: שמספר השניות לא יעלה על אורך השיר - זה דורש הצלבה עם טבלת השירים.

דרישה 3 - איתור אמנים בשגיאות כתיב

בחרתי בתשובה השלילית: השגיאות אינן מיוצגות בבסיס הנתונים.

- מרחב השגיאות אינו חסום: אפשר לכתוב Beyoncé בעשרות דרכים, ולכל שם חדש היה צריך לנחש מראש את שגיאותיו. טבלה כזו לעולם אינה שלמה.
 - רישומן ב-ArtistNames היה מזהם אותה בשורות שאינן שמות אמיתיים, ומי שיבקש את שמות האמן יקבל גם אותן.
 - התאמה מקורבת היא תפקיד מנוע החיפוש. המבנה שומר מה נכון; החיפוש מתמודד עם מה שהוקלד.
- מה כן עושים - בשאילתה ולא בטבלה. LIKE נותן חיפוש חלקי, כלומר מי שהקליד רק חלק מהשם עדיין מוצא:

```
SELECT DISTINCT artist_id
FROM ArtistNames
WHERE artist_name LIKE '%קיר%';
```

חשוב לדייק: זה אינו פתרון לשגיאות כתיב. LIKE עוזר רק כשמה שהוקלד הוא רצף נכון מתוך השם, ושגיאה אמיתית - החלפת אות באמצע המילה - אינה נתפסת בו. התאמה כזו מחייבת השוואת מרחק בין מחרוזות, שנעשית באפליקציה על אותה עמודה שכבר קיימת. הסכמה אינה משתנה בגללה, וזו הסיבה שהתשובה שלילית.

מה לא הצלחנו להגן עליו בבסיס הנתונים

CHECK בודק שורה אחת בכל פעם. הוא אינו סופר שורות אחרות ואינו מסתכל על טבלה אחרת. לכן כל כלל שדורש ספירה או הצלבה נשאר מחוץ לסכמה. אף ש-CHECK אינו המנגנון המתאים לחישובים חוצי-שורות או טבלאות, ניתן ואף רצוי לממש הגנות אלו בשכבת האפליקציה שמזינה את הנתונים, וכן יש לנטר מקרים שבהם חוקים אלו מופרים בפועל.

למה	מה לא מוגן
צריך לספור שורות	שלכל אמן יהיה בדיוק שם ראשי אחד
צריך לספור שורות בטבלה אחרת	שלכל שיר יהיה לפחות אמן ראשי אחד
צריך להצליב מול טבלת השירים	ש-seconds_played לא יעלה על אורך השיר
צריך לספור שורות	שהמיקומים ברשימה יהיו רצף בלי חורים
אין טבלת סוגים ולכן אין מפתח זר; ראו למעלה	שכל שיר יקבל סוג מתוך רשימה סגורה

נורמליזציה: הסכמה עומדת בצורה נורמלית שלישית - שם האמן אינו יושב בשני מקומות, אין עמודה חוזרת, ואין שדה נגזר (למשל אחוז השמעה, שנובע מהשניות ומאורך השיר).

3.3 - כיצד המידע משפר את חוויית השימוש

- דירוג משתמע: היחס בין seconds_played לבין duration_sec הוא ציון בין אפס לאחד לכל זוג מאזין-שיר. זו מטריצת ההעדפות שההמלצות רצות עליה, והיא מלאה בהרבה מזו שהלייקים היו מייצרים.
- שתי נטישות שונות: דילוג בשש השניות הראשונות אומר שהשיר לא התאים מלכתחילה. עצירה בשנייה השבעים אומרת שהוא נכנס ולא החזיק. הראשונה מחייבת שינוי בבחירת השירים, השנייה בסדר שלהם.
- סינון האוטו-פליי: שיר עם שיעור דילוג גבוה יורד מהמלצות אוטומטיות, אבל נשאר בחיפוש - מי שמבקש אותו במפורש עדיין מקבל אותו.
- זיהוי כוכבים: אותו מידע מזין את שאילתת ה-BCG בסעיף הבא. ומיון לפי הסתברות סיום, ולא לפי פופולריות גולמית, מאריך את משך ההאזנה.

הערה על היקף: טבלת ההאזנות גדלה מהר מכל השאר והיא גם מידע אישי, ולכן הייתי שומר את השורות הגולמיות לחלון מוגבל ומעבר לכך רק סיכומים יומיים ברמת השיר.

3.4 - שאילתה שמאתרת כוכבים

במודל BCG כוכב הוא מוצר שנתח השוק שלו גבוה וגם ממשיך לצמוח. המודל אינו קובע ספים, ולכן אני מגדיר במפורש: כוכב הוא שיר שמספר ההאזנות שלו בחודש האחרון גבוה מהממוצע בקטלוג, ושעלה בלפחות 20 אחוז לעומת החודש שקדם. שני הספים ניתנים לכיול ומופיעים כשני התנאים ב-WHERE.

```
WITH recent AS (
    -- החודש האחרון
    SELECT song_id, COUNT(*) AS plays_now
    FROM Plays
    WHERE play_datetime >= DATE_SUB(CURRENT_DATE, INTERVAL 30 DAY)
    GROUP BY song_id
),
previous AS (
    -- החודש שלפניו
    SELECT song_id, COUNT(*) AS plays_before
    FROM Plays
    WHERE play_datetime >= DATE_SUB(CURRENT_DATE, INTERVAL 60 DAY)
    AND play_datetime < DATE_SUB(CURRENT_DATE, INTERVAL 30 DAY)
    GROUP BY song_id
)
SELECT s.song_id, s.song_title, r.plays_now, p.plays_before
FROM recent AS r
JOIN previous AS p ON p.song_id = r.song_id
JOIN Songs AS s ON s.song_id = r.song_id
WHERE r.plays_now > (SELECT AVG(plays_now) FROM recent) -- נתח שוק גבוה
```

```
AND r.plays_now > p.plays_before * 1.2
ORDER BY r.plays_now DESC;
```

וממשיך לצמוד --

שתי הערות:

- ה-JOIN עם previous מוציא מהתוצאה שירים שיצאו החודש ולא היו קיימים קודם. זו החלטה ולא תקלה: שיר חדש שזינק אינו כוכב אלא סימן שאלה, כי עוד לא הוכיח החזקה לאורך זמן. היפוך שני התנאים ב-WHERE מחזיר את שאר הקבוצות של המודל.
- אפשר להחמיר ולספור רק האזנות שהסתיימו, בהוספת 'finished' = end_reason לשתי הקבוצות: שיר שמושמע הרבה ונזנח באמצע אינו הצלחה אמיתית.

3.5 - האם המערכת צריכה לייצג playlists בעלי תוכן זהה

כן, ובכוונה לא מנעתי זאת. רשימת השמעה מזוהה בבעליה ובכוונתה, לא בתוכנה.

- בין מאזינים שונים: שניים שיצרו במקרה את אותם עשרים שירים הם מקרה רגיל לגמרי. חסימה כאן פירושה להשיב למאזין השני "מישהו כבר יצר את זה", התנהגות אבסורדית במוצר.
- אצל אותו מאזין: "אימון" ו"ריצה" יכולים להיות זהים היום ולהתפצל מחר. הזהות מקרית ורגעית.
- מחיר האכיפה: התוכן משתנה בכל הוספה, הסרה ושינוי סדר, ואכיפה הייתה מחייבת חתימה על התוכן שמתעדכנת בכל עריכה - בדיקה כבדה כדי למנוע מצב שאינו שגירה. וממילא "תוכן זהה" אינו מוגדר היטב: האם אותם שירים בסדר אחר הם רשימה זהה?

מה כן ראוי לשים לב אליו: אם אלף מאזינים העתיקו את אותה רשימה, מודל שסופר שירים שמופיעים יחד יסיק שהם קשורים יותר ממה שנכון. הפתרון אינו למנוע את הכפילות אלא לתת לרשימות זהות משקל אחד בשלב הניתוח. לייצג - כן. למנוע - לא. לספור - בהירות.

3.6 בונס - מודל שאפשר לממש ב-Spotify ואי אפשר על IMDB

ההבדל אינו בכמות הנתונים אלא בטיבם. IMDB מחזיק דירוג מפורש - מספר אחד לכל צופה לכל סרט, בלי סדר ובלי צפייה חלקית. Spotify מחזיק התנהגות: אותו שיר נשמע עשרות פעמים, ברצף, ולעיתים באופן חלקי.

המודל: המלצה על השיר הבא לפי מה שבא אחרי מה. כל האזנה של מאזין היא נקודה ברצף, והמעבר משיר לשיר הוא תצפית. ריכוז כל המעברים נותן, לכל שיר, את השיר שהכי סביר שיבוא אחריו. השאלתה שמייצרת את זוגות האימון משתמשת ב-LAG:

```
WITH ordered AS (
  SELECT listener_id, song_id, play_datetime, end_reason,
         LAG(song_id) OVER (PARTITION BY listener_id
                           ORDER BY play_datetime) AS prev_song,
         LAG(end_reason) OVER (PARTITION BY listener_id
                              ORDER BY play_datetime) AS prev_reason
  FROM Plays
)
SELECT prev_song, song_id, COUNT(*) AS transitions
FROM ordered
WHERE prev_song IS NOT NULL
AND prev_reason = 'finished' -- רק מעבר אחרי שיר שהושמע עד הסוף
GROUP BY prev_song, song_id
ORDER BY transitions DESC;
```

התנאי האחרון הוא הלב: מעבר שבא אחרי דילוג אינו מעיד על קשר בין השירים אלא על בריחה מהראשון. בלעדיו המודל לומד רצפים שנוצרו מחוסר סבלנות.

למה זה בלתי אפשרי על IMDB:

- רצף מסודר של צריכה: ב-Spotify לכל האזנה יש זמן ולכן יש רצף. ב-IMDB לדירוגים אין סדר צפייה.
- צריכה חוזרת של אותו פריט: שיר נשמע שוב ושוב, בעוד סרט נצפה פעם אחת ומקבל דירוג אחד.
- אות שלילי משתמע: דילוג אחרי שש שניות הוא תיוג שלילי. ב-IMDB אין רישום של צפייה חלקית.

אופציה נוספת ללא פונ' חלון

המודל: למידה ממשוב שלילי משתמע (Implicit Negative Feedback)

ב-Spotify, משתמשים כמעט ולא מדרגים שירים בצורה מפורשת. עם זאת, המערכת אוספת עליהם מידע התנהגותי רציף. המודל ייצר מטריצת העדפות על בסיס חישוב ציון העדפה (preference_score) לכל זוג מאזין-שיר, הנגזר מ"שיעור הנטישה המיידית" לעומת "שיעור ההשלמה".

הערה: הספים שנבחרו כאן (נטישה מתחת ל-10 שניות, ומינימום 3 האזנות לזוג כדי להיכנס לחישוב) הם הגדרות של בחירת פרמטרים שניתנים לכיול, בדיוק כמו הספים במודל ה-BCG.

שאלתת SQL (יצירת קלט למודל): השאלתה מסכמת לכל מאזין ולכל שיר את סך ההאזנות, ומחשבת את ציון ההעדפה הסופי המשמש כקלט.

```
SELECT
  listener_id,
```

```

song_id,
COUNT(*) AS total_plays,
SUM(CASE WHEN end_reason = 'skipped' AND seconds_played < 10 THEN 1 ELSE 0 END) AS fast_skips,
SUM(CASE WHEN end_reason = 'finished' THEN 1 ELSE 0 END) AS completed_plays,
( SUM(CASE WHEN end_reason = 'finished' THEN 1 ELSE 0 END)
- SUM(CASE WHEN end_reason = 'skipped' AND seconds_played < 10 THEN 1 ELSE 0 END)
) / COUNT(*) AS preference_score
FROM Plays
GROUP BY listener_id, song_id
HAVING COUNT(*) >= 3;

```

השדה preference_score נע בין 1-ל-1: ציון 1 פירושו שכל ההאזנות הושלמו, וציון 1- אומר שכולן ננטשו מיד. מספר זה הוא בדיוק מה שמחליף את הדירוג המפורש של IMDB.

למה זה בלתי אפשרי ב-IMDB?

- היעדר נתוני נטישה: ב-IMDB, אם התחלת לראות סרט וכיבית אותו אחרי 10 דקות כי הוא שעמם אותך, הפלטפורמה לרוב לא יודעת מזה. מודל כזה הוא בלתי אפשרי ליישום כשאין שום רישום של צפייה שנקטעה באמצע.
- צריכה חוזרת ושינוי העדפות: סרט ב-IMDB נצפה ומקבל דירוג אחד קבוע. ב-Spotify, שיר מושמע שוב ושוב לאורך זמן. ב-IMDB גם אין סדר בין הדירוגים, ולכן אי אפשר לזהות שההעדפה השתנתה לאורך זמן (למשל, משתמש שאהב שיר וחרש עליו בעבר, אך כעת מדלג עליו באופן עקבי) - אפשר לראות רק מספר סופי אחד.
- סוג המשוב: IMDB נשען כמעט לחלוטין על דירוג מפורש אקטיבי מיוזמת המשתמש. ב-Spotify הדירוג הוא משתמע ונוצר מעצם התנהגות ההאזנה ללא צורך בפעולה אקטיבית.